

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

INSTITUTO DE CIÊNCIAS EXATAS E INFORMÁTICAS - ICEI

Curso de Graduação em Engenharia De Computação

Campus Coração Eucarístico

Maicon Gomes Messias

TRABALHO TEÓRICO 9

Referencia e Apontadores

BELO HORIZONTE

2020

Maicon Gomes Messias

TRABALHO TEÓRICO 9

Referencia e Apontadores

Trabalho Teórico 9 da disciplina Algoritmos e Estrutura de Dados II, lecionada no curso Engenharia de Computação – 2º Período.

Orientador: Kleber Souza

OUTUBRO

2020

1. Se *i* é um valor inteiro e *p* e *q* são ponteiros para inteiros, quais seguintes atribuições causam erro de compilação?

   a) p = &i;

   b) p = *&i;

   c) p = &*i; erro de compilação

   d) i = *&*p;

   e) i = *&p;

   f) i = &*p;

   g) p = &*&i;

   h) q = *&*p;

   i) q = **&p; erro de compilação

   j) q = *&p;

   k) q = &*p;

2. O que significa o operador asterisco ***** em cada um dos seguintes casos:

   a) int *p; apontando para endereço alocado na memoria

   b) printf("%d", *p); imprime o endereço da memoria

   c) *p = x*5;

   d) printf("%d", *(p+1)); printa o elemento do vetor onde o ponteiro esta apontando.

3. Um vez que as declarações

```
int vetor[] = {1,2,3}, *ptr = vetor;
```

Tenham sido feitas, qual será o conteúdo de *vetor* e *ptr* depois de executar individualmente (não em sequência):

a) *ptr++; Resultado 1,2,3

```
valor ptr=2
vetor[0]=1
vetor[1]=2
vetor[2]=3
```

b) (*ptr)++; Resultado 2,2,3

```
valor ptr=2
vetor[0]=2
vetor[1]=2
vetor[2]=3
```

c) *ptr++; (*ptr)++; Resultado 1,3,3

```
valor ptr=3
vetor[0]=1
vetor[1]=3
vetor[2]=3
```

4. Usando somente ponteiros (sem indexação de matriz), escreva uma função para somar todos os números de uma matriz de inteiros.

```c
#include <stdio.h>
#include <stdlib.h>
#define coluna 3
#define linha 3
int soma_mat(int aux)
{
    int soma=0;
    for(int i=0; i<linha; i++)
    {
        for(int j=0; j<coluna; j++)
        {
            aux++;
            printf("%d ",aux);
            soma=soma+aux;
        }
        printf("\n");
    }
    printf("soma eh: %d",soma);

    return soma;
}
int main()
{
    int aux=0;
    int **ptr =(int *)malloc(linha * sizeof(int));
    for(int i=0; i<coluna; i++)
    {
        *(ptr+i)=(int *)malloc(coluna * sizeof(int));
    }
    for(int i=0; i<linha; i++)
    {
        for(int j=0; j<coluna; j++)
        {

            *(*(ptr+i)+j)= aux;
```

```
        }
    }
    soma_mat(aux);

    return 0;
}
```

5. Usando somente ponteiros, implemente as seguintes funções de cadeias de caracteres:

   a) strlen()

```
#include <stdio.h>
#include <stdlib.h>

int tam_strlen(char *palavra)
{
    int cont = 0;
    while(*palavra!='\0')
    {
        cont++;
        *palavra++;
    }
    return cont;
}

int main()
{
    int tamanho;
    char *palavra[30];
    printf("digite a palavra: ");
    scanf(" %[^\n]s\n",palavra);
    tamanho = tam_strlen(&palavra);
    printf("A PALAVRA: %s TEM %d LETRAS",palavra,tamanho);
    return 0;
}
```

   b) strcmp()

   c) strcat()

   d) strchr()

6. Qual diferença das duas instruções abaixo?

```
if (p == q) { ... }

if (*p == *q) { ... }
```

A diferença é que o primeiro *if* faz a comparação de valores que estão armazenados na variável p e q. Já no segundo *if* realiza a comparação de endereço, ou seja, se a variável p e q estão apontando para o mesmo endereço de memória.

7. Usando ponteiro para função, crie um programa que simule uma calculadora com as operações de soma, subtração, divisão e multiplicação. A operação deve ser executada em dois números inteiros e deve ser alocada dinamicamente em um ponteiro após o usuário definir qual operação deseja executar.

```
#include <stdio.h>
#include <stdlib.h>
void calculadora(int *a,int *b)
{
	int opc = -1;
	int soma;
	int subtracao;
	int mult;
	int divisao;

	while(opc != 0)
	{
		printf("\n-----informe a operacao desejada-----\n");
		printf("1-soma\n2-subtracao\n3-multiplicacao\n4-divisao\n");
		scanf("%d",&opc);
		switch ( opc )
		{
		case 1 :
		soma =(int *)malloc(1);
		soma=(*a)+(*b);
		printf ("soma eh: %d\n",soma);
		break;
		case 2 :
		subtracao=(int *)malloc(1);
		subtracao=(*a)-(*b);
		printf ("subtracao eh:%d\n",subtracao);
		break;
		case 3 :
		mult=(int *)malloc(1);
		mult=(*a)*(*b);
		printf ("multiplicacao eh:%d\n",mult);
		break;

		case 4 :
		divisao=(int *)malloc(1);
		divisao=(*a)/(*b);
		printf ("divisao eh:%d\n",divisao);
		break;
```

```
            default :
            printf ("Valor invalido!\n");
            }

      }
            free(soma);
            free(subtracao);
            free(mult);
            free(divisao);
}
int main()
{
      int a;
      int b;
      printf("digite o primeiro valor:");
      scanf("%d",&a);
      printf("digite o segundo valor:");
      scanf("%d",&b);
      calculadora(&a,&b);
      return 0;
}
```

8. Faça um programa que leia um valor *n* e crie dinamicamente um vetor de *n* elementos. Preencha o vetor com *n* elementos aleatórios e em seguida apresente a soma dos elementos do vetor.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void soma_vetor(int ptr[],int *n,int *soma)
{
      //laco de repeticao pra gerar numero aleotorio
      for(int j=0;j<*n;j++)
      {
            ptr[j]= rand() % 10;
      }
      //=========================================
//laco de repeticao pra printar os valores aleotorio
      for(int j=0;j<*n;j++)
      {
            printf("%d ",ptr[j]);
      }
      //=========================================
      //laco de repeticao pra somar os valores do vetor.
      for(int j=0;j<*n;j++)
      {
            *soma=*soma + ptr[j];
      }
```

```
	printf("\nsoma eh: %i",*soma);
	free(ptr);
}
int main()
{
	int *ptr,n,soma=0;
	scanf("%d",&n);
	printf("\n ");
	ptr=(int *)malloc(1*sizeof(int));
	soma_vetor(ptr,&n,&soma);
	return 0;
}
```

9. Construa um programa que aloque em tempo de execução (dinamicamente) uma matriz de ordem *m* x *n* (linha por coluna). Preencha a matriz com elementos aleatórios e em seguida exiba o menor número e a sua posição na matriz, o maior número e a sua posição na matriz

```
#include <stdio.h>
#include <stdlib.h>
#define coluna 3
#define linha 3

void matriz_menor_maior(int **mat)
{
	int menor;
	int maior=0;
	int j2=0;
	int i2=0;
	int j1=0;
	int i1=0;
	for(int i=0; i<linha; i++)
	{
		for(int j=0; j<coluna; j++)
		{
		*(*(mat+i)+j)=( rand() % 11);
		printf("%d ",*(*(mat+i)+j));
		if(*(*(mat+i)+j)>maior)
		{
			maior = *(*(mat+i)+j);
			i1=i;
			j1=j;
		}
			if(*(*(mat+i)+j)<menor)
		{
```

```
                menor = *(*(mat+i)+j);
                i2=i;
                j2=j;
            }

            }

            printf("\n");
        }

        printf("Maior eh: %d com a posicao %d|%d\n",maior,i1,j1);
        printf("Menor eh: %d com a posicao %d|%d",menor,i2,j2);
}
int main()
{
        int aux=0;
        int **mat =(int *)malloc(linha * sizeof(int));
        for(int i=0; i<coluna; i++)
        {
            *(mat+i)=(int *)malloc(coluna * sizeof(int));
        }
        for(int i=0; i<linha; i++)
        {
            for(int j=0; j<coluna; j++)
            {

            *(*(mat+i)+j)= aux ;
            }
        }
        matriz_menor_maior(mat);

        return 0;
}
```

Crie um programa para gerenciar o cadastro de produtos. Cada produto deve ser armazenado em uma estrutura com as informações de **código**, **descrição** e **valor**. Seu programa deve ser capaz de armazenar *n* produtos, sendo que o valor *n* deve ser informado pelo usuário. Seu programa deve usar funções e ponteiros para executar as seguintes operações:

- Cadastrar um novo produto
- Editar os dados de um produto
- Excluir um produto
- Exibir todos os produtos cadastrados
- Exibir a quantidade total de produtos